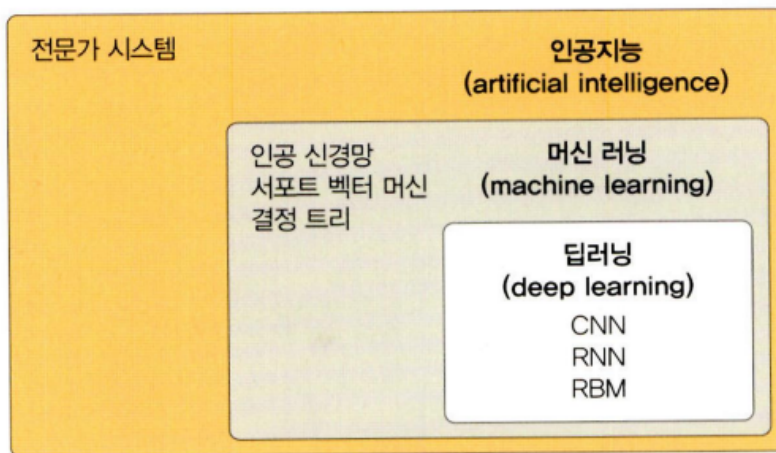


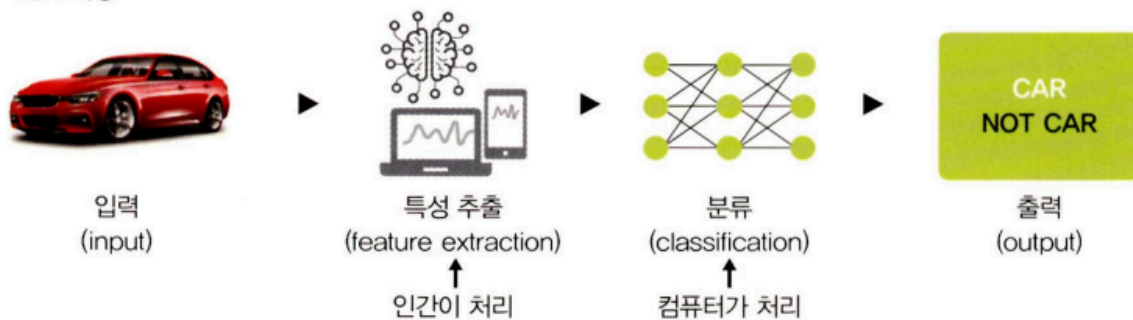
<https://jade0728.tistory.com/60> 에서 확인가능합니다.

1. 머신러닝과 딥러닝

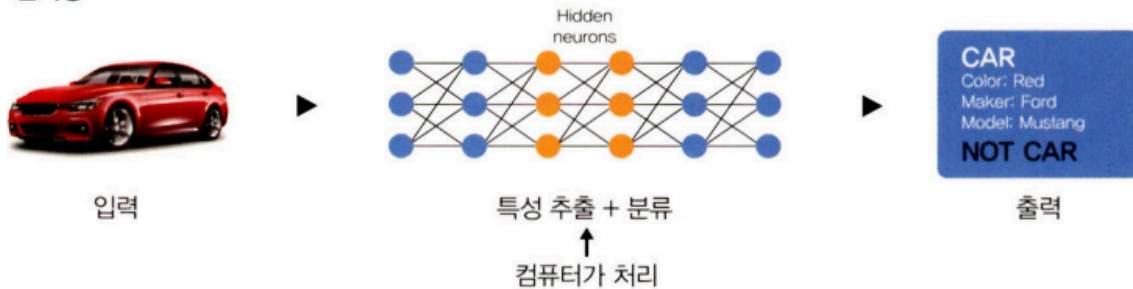
1.1 인공지능, 머신러닝, 딥러닝



머신 러닝



딥러닝



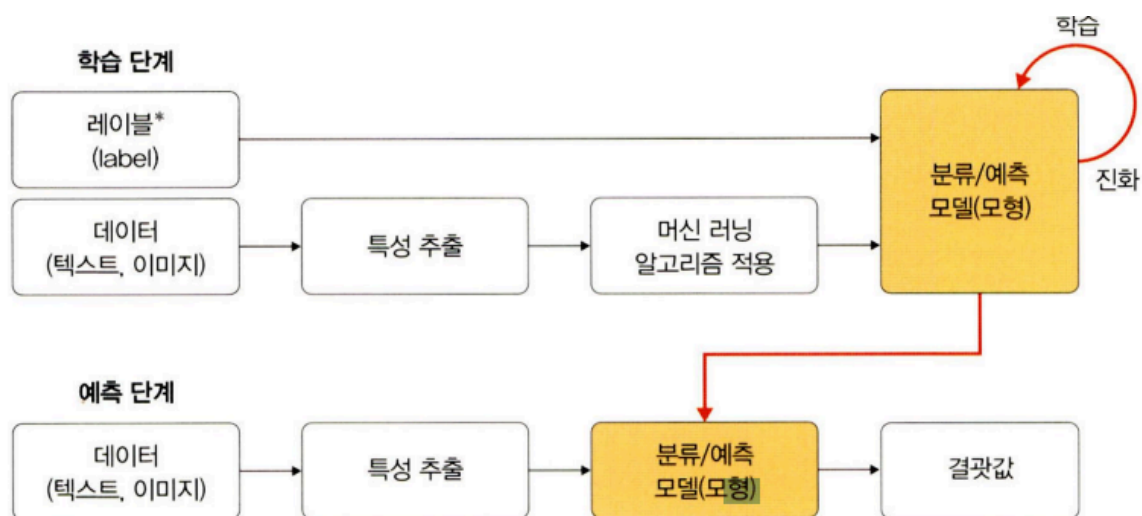
머신러닝은 입력데이터에 알고리즘을 적용하여 예측을 수행한다. 데이터로부터 특징을 추출하는 과정은 인간이 수행한다. 일반적으로 수천개의 데이터가 필요하고, 단시간에 훈련시킬 수 있다.

딥러닝은 정보를 전달하는 신경망을 사용하여 데이터 특징 및 관계를 해석한다. 구현된 알고리즘은 동일한 유형의 데이터를 분석하는데 재사용된다. 보통 수백만 개 이상의 데이터가 필요하고 장시간에 걸쳐 훈련시켜야한다.

1.2 머신러닝이란?

1.2.1 머신러닝 학습과정

학습단계(**learning**) 과 예측단계(**prediction**)으로 구분된다. 훈련 데이터를 머신 러닝 알고리즘에 적용하여 학습시키고, 이 학습결과로 모형이 생성된다. 예측단계에서는 생성도니 모형에 새로운 데이터를 넣어서 결과를 예측한다.



* 레이블은 지도 학습에서 정답을 의미

1.2.2 머신러닝 학습 알고리즘

머신러닝의 학습 알고리즘으로는 지도학습, 비지도학습, 강화학습이 존재

지도학습: 정답이 무엇인지 컴퓨터에 알려주고 학습하는 방법이다.

비지도 학습: 정답을 알려주지 않고 비슷한 데이터를 클러스터링하여 예측

강화 학습: 자신의 행동에 대한 보상을 받으며 학습 진행

1.3 딥러닝이란

1.3.1. 딥러닝 학습 과정

딥러닝 모델의 학습과정은 데이터 준비 → 모델 정의 → 모델 컴파일 → 모델 훈련 → 모델 예측 으로 구성된다.

- 1) 데이터 준비: 캐글, 파이토치, 케라스 등에서 데이터를 구할 수 있다. 필요시 전처리 과정을 거쳐야 할 수 있다.
- 2) 모델 정의: 신경망을 생성한다. **hidden layer**가 많을 수록 성능이 좋아지지만 **overfitting**이 발생할 수도 있다.
- 3) 모델 컴파일: **activation** 함수, **loss** 함수, **optimizer** 를 선택한다. **MSE**(평균 제곱 오차), 이진 분류, **cross entropy** 등의 방법을 선택할 수 있다.
- 4) 모델 훈련: 훈련 단계에서는 한번에 처리할 데이터 양을 선택한다. 한번에 처리해야 하는 데이터 양이 많아지면 학습 속도가 느려지고 메모리가 부족해질 수 있어 적절한 데이터양으로 선택하는 것이 중요하다. 따라서 **batch**, **epoch** 선택이 중요하다. 그리고 파라미터와 하이퍼파라미터의 최적값을 찾아야한다.
- 5) 모델 예측: 새로운 데이터셋을 모델에 넣어서 실제로 예측을 진행해본다.

1.3.2. 딥러닝 학습 알고리즘

지도학습, 비지도학습, 전이학습으로 분류된다.

먼저 지도학습 방법에 대해 알아보자.

지도학습으로는 컴퓨터비전의 합성곱 신경망(**Convolutional Neural Network, CNN**)이 있다.

합성곱 신경망은 목적에 따라 이미지 분류, 이미지 인식, 이미지 분할로 분류할 수 있다.

이미지 분류는 이미지를 알고리즘에 입력하면 그 이미지가 어떤 클래스에 속하는지 알려주기 때문에 이미지 데이터를 유사한 것끼리 분류할 때 사용한다. 이미지 인식은 사진을 분석하여 사진 속 사물의 종류를 인식할 때 사용한다. 마지막으로 이미지 분할은 영상에서 객체 간 영역을 픽셀 단위로 구분하는 기술이다.

지도학습의 또다른 방법으로는 순환 신경망(**RNN**)과 RNN의 **gradient vanishing**을 개선한 **LSTM**이 있다.

비지도 학습으로는 워드 임베딩과 군집이 있다.

워드 임베딩은 자연어를 컴퓨터가 이해할 수 있도록 단어를 벡터로 표현하는 기술이다.

워드 임베딩에서는 Word2Vec과 GloVe 기술을 사용한다.

군집은 아무런 정보가 없는 상태에서 데이터를 분류하는 방법이다.

전이학습(**transfer learning**)은 사전에 학습이 완료된 모델을 가지고 우리가 원하는 학습에 미세 조정 기법을 이용하여 학습시키는 방법이다. 사전학습모델은 풀고자 하는 문제와 비슷하면서도 많은 데이터로 이미 학습이 되어 있는 모델이다. 보통 VGG, Inception, MobileNet 같은 모델을 사용한다

구분	유형	알고리즘
지도 학습(supervised learning)	이미지 분류	• CNN • AlexNet • ResNet
	시계열 데이터 분석	• RNN • LSTM
비지도 학습(unsupervised learning)	군집(clustering)	• 가우시안 혼합 모델(Gaussian Mixture Model, GMM) • 자기 조직화 지도(Self-Organizing Map, SOM)
	차원 축소	• 오토인코더(AutoEncoder) • 주성분 분석(PCA)
전이 학습(transfer learning)	전이 학습	• 버트(BERT) • MobileNetV2
강화 학습(reinforcement learning)	—	마르코프 결정 과정(MDP)

2. 파이토치

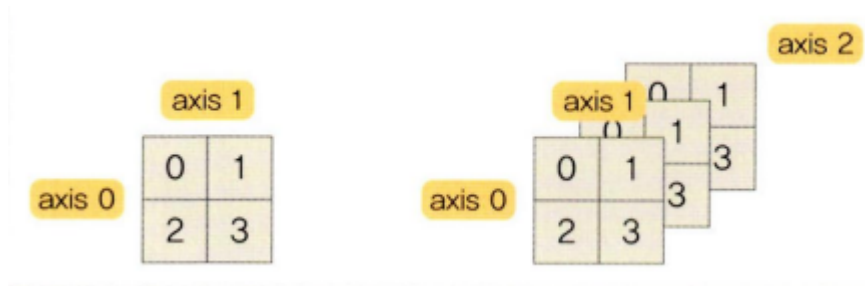
2.1 파이토치 개요

벡터, 행렬, 텐서

행=1차원 축= axis 0 = 벡터

열=2차원 축= axis 1 = 행렬

채널=3차원 축= axis 2 = 텐서



파이토치 **api**

torch: GPU 기반 텐서 패키지

torch.autograd: 자동 미분 패키지

torch.nn: 신경망 구축 및 훈련 패키지

torch.multiprocessing: 파이썬 멀티프로세싱 패키지

torch.utils : 기타 유틸리티

2.2 파이토치 기초 문법

2.2.1 텐서 및 데이터 준비

텐서 생성

```
import torch
```

```
print(torch.tensor([1,2],[3,4]))
```

텐서를 **ndarray**로 변환

```
temp= torch.tensor([[1,2],[3,4]])
```

```
print(temp.numpy())
```

데이터 준비: 파일 불러오기

```
pip install pandas
```

```
import pandas as pd
```

```
import torch
```

```
data=pd.read_csv('../class2.csv')
```

데이터 준비: 커스텀 데이터셋

```
import pandas as pd
```

```
import torch
```

```
from torch.utils.data import Dataset
```

```
from torch.utils.data import DataLoader
```

```
class CustomDataset(Dataset):
```

```

def __init__(self, csv_file): # csv_file을 통해 데이터셋 불러오기
    self.label = pd.read_csv(csv_file)

def __len__(self): # 전체 데이터셋의 크기를 반환
    return len(self.label)

def __getitem__(self, idx): # 전체 x, y 데이터 중 해당 인덱스의 데이터를 가져오기
    sample=torch.tensor(self.label.iloc[idx, 0:3]).int()
    label=torch.tensor(self.label.iloc[idx, 3]).int()

    return sample, label

```

데이터 준비: 파이토치에서 제공하는 데이터셋 사용

```
pip install requests
```

2.2.2 모델 정의

pytorch에서는 모델을 정의하기 위해 모듈을 상속한 클래스를 사용

- 계층: 모듈 또는 모듈을 구성하는 한 개의 계층
- 모듈: 한 개 이상의 계층이 모여서 구성된 것
- 모델: 최종적으로 원하는 네트워크]

단순 신경망 정의하기

```
model=nn.Linear(in_features=1, out_features=1, bias=True)
```

nn.Module()을 상속하여 정의하기

-> 기본적으로 **__init__()**, **forward()** 함수를 제공

```

class MLP(Module):
    def __init__(self, inputs):
        super(MLP, self).__init__()
        self.layer=Linear(inputs, 1) # 계층 정의
        self.activation=Sigmoid() # 활성화 함수 정의

    def forward(self, X):
        X= self.layer(X)
        X= self.activation(X)

    return X

```

Sequential 신경망을 정의하기

-> **__init__()** 제공, **forward()** 함수에서 해야하는 계산을 더 가독성있게 작성 가능하다

```

import torch.nn as nn
class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.layer1= nn.Sequential(
            nn.Conv2d(in__channels=3, out_channels=64, kernel_size=5),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2))

        self.layer2= nn.Sequential(
            nn.Conv2d(in__channels=64, out_channels=30, kernel_size=5),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2))

        self.layer3= nn.Sequential(
            nn.Conv2d(in__channels=30*5*5, out_channels=10, bias=True),
            nn.ReLU(inplace=True))

    def forward(self , x):
        x=self.layer1(x)
        x=self.layer2(x)
        x=x.view(x.shape[0], -1)
        x=self.layer3(x)
        return x

model= MLP() # 모델에 대한 객체 생성

```

2.2.3 모델의 파라미터 정의

1) 손실함수: loss function

주로 사용되는 함수는 BCELoss(이진 분류에 사용), CrossEntropyLoss(다중 클래스 분류를 위해 사용), MSELoss(회귀 모델에서 사용)

2) optimizer: 데이터와 손실 함수를 바탕으로 모델의 업데이트 방법을 결정

step() 메서드를 통해 전달받은 파라미터를 업데이트

torch.optim.Optimizer(params, defaults): optimizer의 기본이 되는 클래스

zero_grad(): optimizer에 사용된 파라미터들의 기울기를 0으로 만듦

torch.optim.lr_scheduler: epoch에 따라 학습률 조절

3) 학습률 스케줄러: learning rate scheduler

epoch를 지날 때마다 학습률을 감소시킨다. global minimum 근처에 다다르면 학습률을 줄여서 최적점에 찾아갈 수 있도록 한다.

2.2.4 모델 훈련 Train

모델 훈련은 $y=wx+b$ 라는 함수의 w, b 의 최적값을 찾는다는 말이다. **loss**가 최소가 될때까지 (w,b) 를 계속 수정해간다.

훈련방법은 다음과 같다.

1) **optimizer.zero_grad()** : 기울기를 초기화

2) **loss.backward()** : 기울기를 자동 계산

```
for epoch in range(100):
    yhat=model(x_train)
    loss=criterion(yhat, y_train)
    optimizer.zero_grad() # 오차가 쌓이지 않도록 초기화
    loss.backward()

    optimizer.step()
```

2.2.5 모델 평가

주어진 테스트 데이터셋을 통해 모델을 평가한다. **pip**로 다음 패키지를 설치해야 모델을 평가할 수 있다.

```
pip install torchmetrics

import torch
import torchmetrics

preds= torch.randn(10, 5).softmax(dim=1)
target= torch.randint(5, (10, ))

acc= torchmetrics.functional.accuracy(preds, target)
```

2.2.6 훈련과정 모니터링

tensorboard를 사용해 각종 파라미터 값이 어떻게 변화하는지 시각화하여 파악할 수 있다.

1) 텐서보드 설정(set up)

2) 텐서보드 기록(write)

3) 텐서보드를 사용해 모델 구조를 파악

```
pip install tensorboard

import torch
```



```
from torch.utils.tensorboard import SummaryWriter

writer= SummaryWriter("../chap02/tensorboard") #모니터링에 필요한 값들이
저장될 위치

for epoch in range(100):
    model.train()
    batch_loss=0.0

    for i, (x,y) in enumerate(dataloader):
        x, y= x.to(device).float(), y.to(device).float()
        outputs=model(x)
        loss=criterion(outputs, y)
        writer.add_scalar("Loss", loss, epoch) # 오차를 기록
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

writer.close()
```